



Unit 1 Introduction to Data Structures

Syllabus	Basic Terminology, Classification of Data Structure-Primitive and Non Primitive Data Structure, Linear and Non-Linear Data Structures, Examples of Primitive and Non Primitive Data Structures, storage representation of Primitive and Non Primitive Data Structures
-----------------	---

Basic Terminology

➤ What is Data Structure?

- The term data means a value or set of values. It specifies either the value of a variable or a constant (e.g., marks of students, name of an employee, address of a customer, value of pi, etc.).
- A **data structure** is a storage that is used to store and organize data. It is a way of arranging data on a computer so that it can be accessed and updated efficiently.
- A data structure is not only used for organizing the data. It is also used for processing, retrieving, and storing data.
- There are different basic and advanced types of data structures that are used in almost every program or software system that has been developed.

➤ Importance of Data Structure?

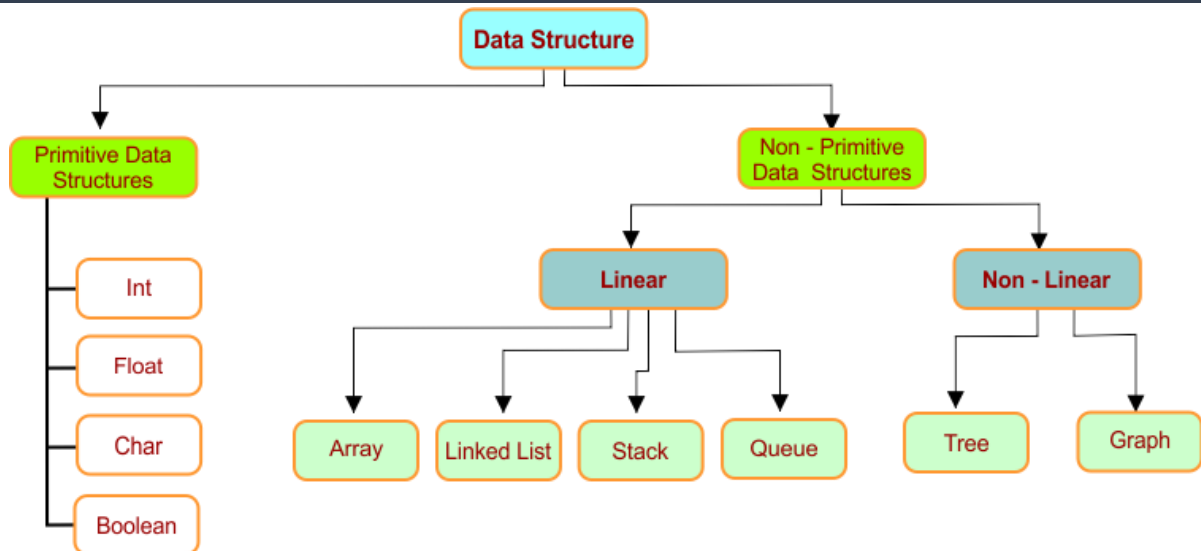
- It helps in improving data storing and organizing.
- It aids in faster retrieval and manipulation of data.
- It also facilitates the designing of algorithms for solving complex problems.
- It helps in data updating and maintaining tasks much more straightforward.
- It is the best solution for understanding the relationship between data and the other elements.

Classification of Data Structures

- Data structures are generally categorized into two classes: primitive and non-primitive data structures.

➤ Primitive and Non-Primitive Data Structures

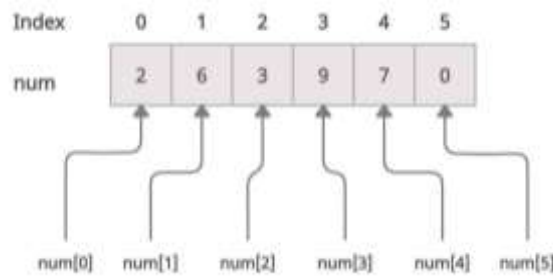
- **Primitive data structures** are the fundamental data types which are supported by a programming language.



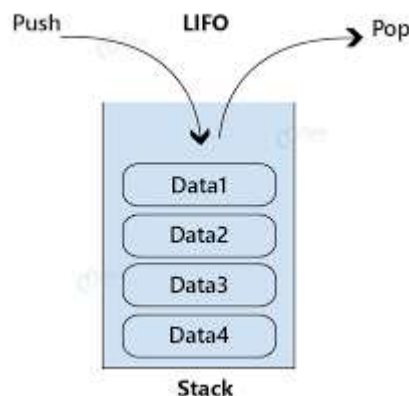
- Some basic data types are integer, real, character, and boolean.
 - Integer: It is used to represent whole numbers (positive or negative).
 - Float: It is used to represent numbers with decimals or floating-point numbers.
 - Character: It represents individual characters, usually stored as a single byte.
 - Boolean: It represents a value of either true or false.
 - String: It represents a sequence of characters.
- **Non-primitive data structures** are those data structures which are created using primitive data structures. They can hold collections of data
- Examples of such data structures include linked lists, stacks, trees, and graphs. Non-primitive data structures can further be classified into two categories: linear and non-linear data structures.

➤ Linear and Non-linear Structures

- **Linear Data Structure**: A data structure that maintains a linear relationship among its elements is called a linear data structure. Here, the data is arranged in a linear fashion. But in the memory, the arrangement may not be sequential. Each element is attached to its previous and next adjacent elements. Ex: Arrays, linked lists, stacks, queues.
- **Array**: An array is defined as the collection of similar type of data items stored at contiguous memory locations.
 - It is a static data structure with a fixed size.
 - Types of Array: One-Dimensional Array (1D Array), Two-Dimensional Array (2D Array), Multi-Dimensional Array (nD Array)
 - Applications of Array : Storing and accessing data, Searching data item, Matrices, Implementing other data structures



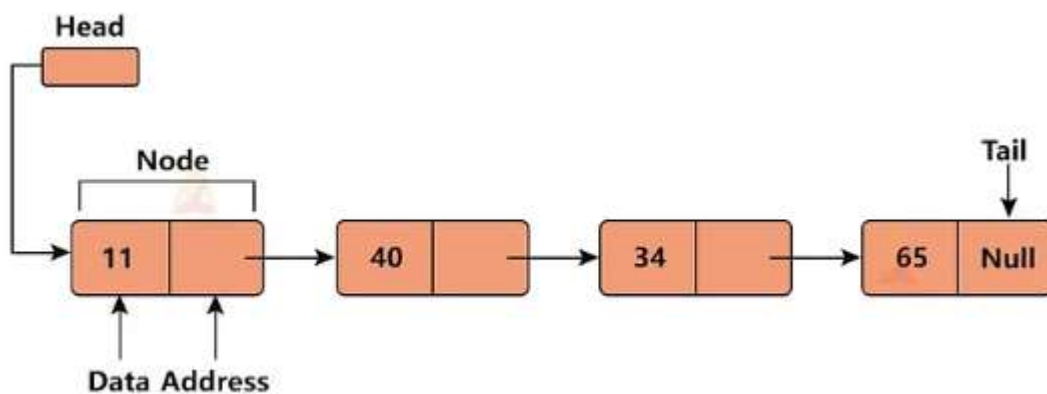
- Stack: Stack is a Linear Data structure in which, insertion and deletion operations are performed at one end only. Stack is also called as Last in First out (LIFO) data structure. The insertion operation is referred to as 'PUSH' and deletion operation is referred to as 'POP' operation.
 - Applications of Stack: Used in Recursion, Undo/Redo operation, Forward Backward navigation in web browsers, converting infix expressions to postfix or prefix.



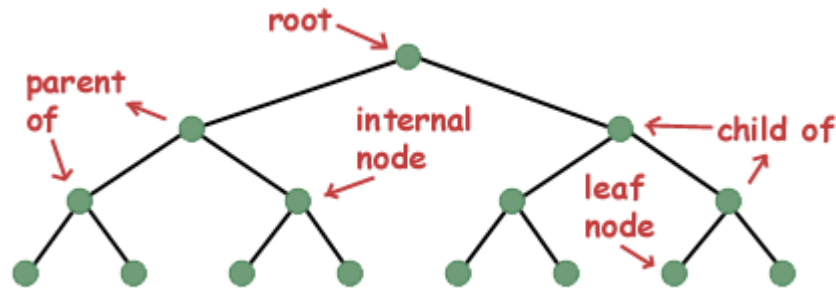
- Queue: The data structure which allow the insertion at one end and Deletion at another end, known as Queue. End at which deletion is occurs is known as FRONT end and another end at which insertion occurs is known as REAR end. Queue is also called as First in First out (FIFO) data structure.
 - Types of Queue: Simple Queue (Linear Queue), Circular Queue, Double-Ended Queue (Deque), Priority Queue, Circular Deque
 - Applications of Queue: CPU scheduling in operating system, printer queue management, handling requests in web servers, ticket booking, data packet transmission in networking



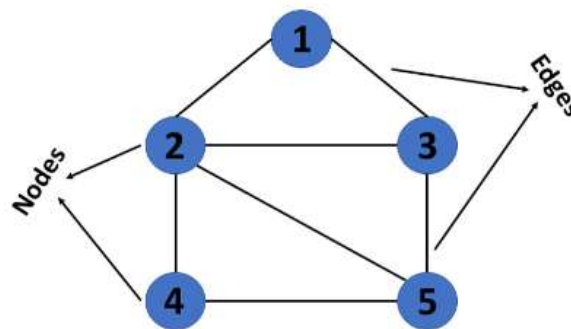
- **Linked list:** Linked List is a linear data structure. Unlike arrays, linked list elements are not stored at a contiguous location the elements are linked using pointers.
 - **Types of Linked List:** Singly Linked List, Doubly Linked List, Circular Linked List, Circular Doubly Linked List
 - **Applications of Linked List:** Navigation systems, media players, and image viewers, used in implementing other dynamic data structures like stacks, queues, Deque, and graphs.



- **Non-Linear Data Structure:** Nonlinear data structures are those data structure in which data items are not arranged in a sequence. There is a hierarchical relationship between individual data items. Here, the insertion and deletion of data is not possible in a linear fashion. Trees and graphs are examples of non-linear data structures.
- **Tree:** A tree can be defined as finite set of data items (nodes) in which data items are arranged in branches and sub branches according to requirement.
 - **Types of Tree:** General Tree, Binary Tree, Full Binary Tree, Binary Search Tree (BST), Red-Black Tree, B Tree etc.
 - **Applications of Tree:** used in representing hierarchical data such as file systems, organizational structures, and XML/HTML document models, parsing expressions in compilers, decision-making algorithms, network routing algorithms



- Graph: Graph is a collection of nodes (Information) and connecting edges (Logical relation) that represent relationships between them.
 - Types of Graph: General Tree, Binary Tree, Full Binary Tree, Binary Search Tree (BST), Red-Black Tree, B Tree etc.
 - Applications of Graph: Google Maps and GPS systems for shortest path finding and route optimization, web page ranking, resource allocation in operating system



➤ Difference between Primitive and Non-Primitive Data Structures

Primitive Data Structures	Non-Primitive Data Structures
These are the most basic data types in programming languages.	This is complex and used for one or more primitive data types.
It is used to represent simple values such as integers, booleans, and characters.	It is used to represent more complex data objects such as arrays, queues, trees, and stacks.
They have a fixed size and range of values.	Non-primitive structures can be resized or modified during runtime.
These data types are predefined in the programming language.	These are usually defined or created by the programmer based on needs.
Primitive data types are generally immutable, meaning their value cannot be changed once created.	Non-primitive data structures are mutable, meaning their contents can be modified.



New L.J Institute of Engineering and Technology
Semester: III Subject: Data Structures (BE03000081)
Unit 1: Introduction to Data Structures

Primitive data types are usually stored in the stack memory.	Non-primitive data structures are typically stored in heap memory.
It uses basic operations such as addition, subtraction, etc.	It uses complex operations such as traversal, sorting, and searching.
It is useful for small and simple programs.	It is used for handling large sets of data, and efficient algorithms.

➤ **Difference between Linear and Non-Linear Data Structures**

Linear Data Structures	Non-Linear Data Structures
Data elements are arranged in a sequential, linear order.	Data elements are arranged hierarchically or in a network.
Example: Arrays, Linked Lists, Stacks, Queues	Example: Trees, Graphs
Data is accessed sequentially. Random access is possible in arrays.	Data is accessed based on relationships or levels.
Traversal is linear; start at one end and follow through to the other end.	Traversal can be complex, such as depth-first or breadth-first in trees and graphs.
Simple operations but may involve shifting elements (except in linked lists).	Can be complex, depending on the position within the structure (e.g., balancing a tree).
Suitable for simple data storage and retrieval where data naturally forms a sequence.	Suitable for representing complex relationships and hierarchical structures.
Operations generally require shifting or linear traversal, which can be inefficient.	Efficiency depends on the type and complexity of the operations, often optimized with algorithms like balancing or path finding.
Typically easier to implement and understand.	Often requires more complex algorithms and understanding of the data connections.

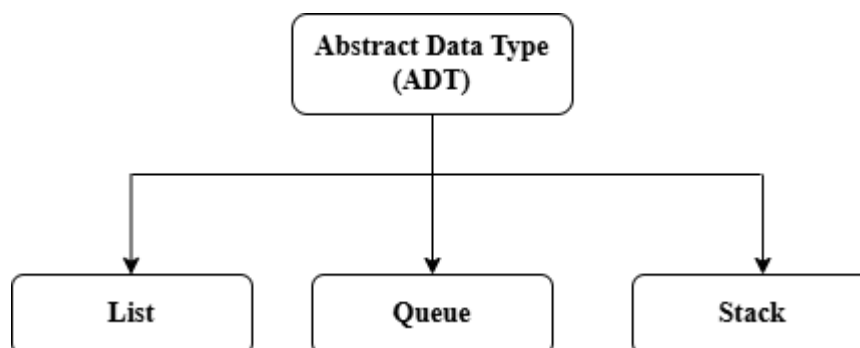
➤ **Differentiate between data types and data structures**

Data Types	Data Structures
Data Type is the kind or form of a variable which is being used throughout the program. It defines that the particular variable will assign the values of the given data type only	Data Structure is the collection of different kinds of data. That entire data can be represented using an object and can be used throughout the entire program.

Implementation through Data Types is a form of abstract implementation	Implementation through Data Structures is called concrete implementation
Can hold values and not data, so it is data less	Can hold different kind and types of data within one single object
Values can directly be assigned to the data type variables	The data is assigned to the data structure object using some set of algorithms and operations like push, pop and so on.
No problem of time complexity	Time complexity comes into play when working with data structures
Examples: int, float, double	Examples: stacks, queues, tree

➤ Abstract data type

- Data types specify the type of data structure.
- We can classify the data type into primitive data types like integer, float, double, etc., or abstract data types like list, stack, queue, etc.
- **Abstract data type (ADT)** is a **concept or model** of a data type.
- It does not specify how data will be organized in memory and what algorithms will be used for implementing the operations.
- It is called “abstract” because it gives an implementation-independent view.



• Features of ADT

- Abstract data types (ADTs) are a way of encapsulating data and operations on that data into a single unit. Some of the key features of ADTs include:
 - **Abstraction:** The user does not need to know the implementation of the data structure only essentials are provided.
 - **Better Conceptualization:** ADT gives us a better conceptualization of the real world.
 - **Robust:** The program is robust and has the ability to catch errors.



- **Encapsulation:** ADTs hide the internal details of the data and provide a public interface for users to interact with the data. This allows for easier maintenance and modification of the data structure.
- **Data Abstraction:** ADTs provide a level of abstraction from the implementation details of the data. Users only need to know the operations that can be performed on the data, not how those operations are implemented.
- **Data Structure Independence:** ADTs can be implemented using different data structures, such as arrays or linked lists, without affecting the functionality of the ADT.
- **Information Hiding:** ADTs can protect the integrity of the data by allowing access only to authorized users and operations. This helps prevent errors and misuse of the data.
- **Modularity:** ADTs can be combined with other ADTs to form larger, more complex data structures. This allows for greater flexibility and modularity in programming.

Operations on Data Structures

- **Traversing-** It is used to access each data item exactly once so that it can be processed.
- **Searching-** It is used to find out the location of the data item if it exists in the given collection of data items.
- **Inserting-** It is used to add a new data item in the given collection of data items.
- **Deleting-** It is used to delete an existing data item from the given collection of data items.
- **Sorting-** It is used to arrange the data items in some order i.e. in ascending or descending order in case of numerical data and in dictionary order in case of alphanumeric data
- **Merging-** It is used to combine the data items of two sorted files into single file in the sorted form

Storage representation of Primitive and Non Primitive Data Structures

➤ Storage representation of Primitive Data Structures

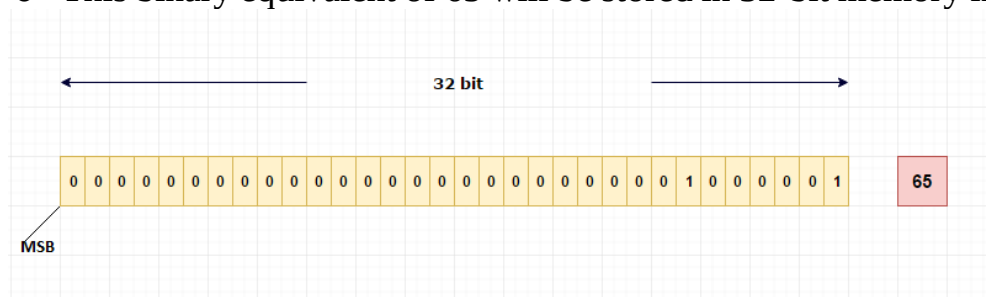
- Primitive data structures are typically stored in memory as binary representations of their values, with each data type having a specific format and size determined by the programming language and hardware architecture.
- **Integer Data Types (int, byte, short, long) :**
 - Integers are typically stored using a fixed number of bits, determined by

the data type.

- The binary representation of the integer value is stored in memory using binary digits (bits).
- For example, an 'int' data type may be stored using 32 bits, allowing for values ranging from (-2^{31}) to $(2^{31}) - 1$.
- Negative integers are typically stored using two's complement representation.
- Memory is allocated in contiguous blocks, with each block containing the binary representation of the integer value.

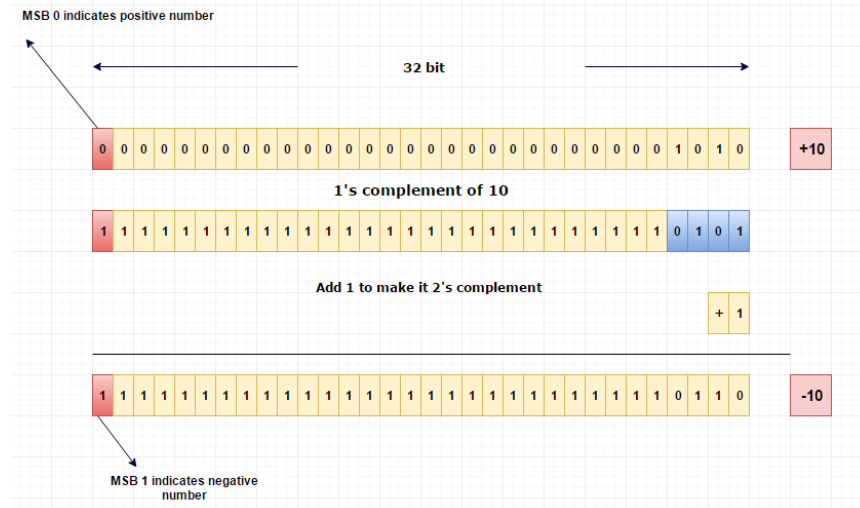
- **Example: 65**

- Binary equivalent of 65 is $(1000001)_2$.
- The MSB (most significant bit) bit is used to indicate whether the number is positive or negative.
- For positive numbers MSB will be 0.
- For negative numbers MSB will be 1.
- In our case, 65 is positive so MSB will be 0.
- This binary equivalent of 65 will be stored in 32-bit memory like below,



- **How negative integers are stored in memory?**

- Computer uses special mechanism to store negative numbers which is **2's complement format**.
- Before going to that, let's have a look on 1's complement of a number.
- **1's complement of a number**
- 1's complement of number is just inverting binary bits of an actual number.
- **Example: 10**
- Binary representation will be $(1010)_2$
- 1's complement of 10 is, 0101 (switching 0 to 1 and 1 to 0).
- 2's complement of a number
- To get 2's complement of a number, just add 1 to 1's complement of an actual number.



- **Floating-Point Data Types (float, double)**

- Floating-point numbers are stored using a binary representation based on the IEEE 754 standard.
- The binary representation consists of three components: sign bit, exponent, and significand (or mantissa).
- The sign bit indicates whether the number is positive or negative.
- The exponent represents the scale factor applied to the significand.
- The significand represents the fractional part of the number.
- Memory is allocated in contiguous blocks, with each block containing the binary representation of the floating-point value.

- **Character Data Type (char)**

- When character value is stored in computer, it is stored with corresponding ASCII value.
- For example, if we want to store char 'A' in computer, the corresponding ASCII value will be stored in computer. ASCII value for capital A is 65.
- To store character value, computer will allocate 1 byte (8 bit) memory. 65 will be converted into binary form which is (1000001)₂.
- Because computer knows only binary number system. Then 1000001 will be stored in 8-bit memory.

	8 Bit memory	ASCII value
A	0 1 0 0 0 0 0 1	65
a	0 1 1 0 0 0 0 1	97



- **Boolean Data Type (boolean)**

- Boolean values are typically stored using a single bit, where 0 represents false and 1 represents true.
- Memory is allocated in contiguous blocks, with each block containing the single bit representing the boolean value

➤ **Storage representation of Non - Primitive Data Structures**

- Non-primitive data structures are more complex data types that are built using primitive data types (like integers, characters, and booleans) or other non-primitive types. They are designed to store and manage collections of data or objects, offering more sophisticated ways to organize and manipulate information.
- Here's how non-primitive data structures are typically represented in storage.
- Storage examples by data structure type:

- **Arrays:** Store elements of the same data type in contiguous memory locations. They can be dynamically allocated, and their size can be changed at runtime in many languages. All elements are of the **same type** and stored **sequentially**. **Indexing** is done using base address + offset.

- Example:

```
int arr[3] = {10, 20, 30};
```

Stored as:

Address	Value
---------	-------

1000	10
------	----

1004	20
------	----

1008	30
------	----

(assuming int takes 4 bytes)

- **Linked Lists:** Consist of nodes, where each node contains data and a pointer (or reference) to the next node in the sequence. This allows for dynamic memory allocation, and the nodes can be stored in non-contiguous memory locations. Also stack and queue can be implemented by Linked Lists.
- Each node has:
 - **Data field** (stores actual value)
 - **Pointer field** (stores address of next node)
- Example Node:

```
struct Node {  
    int data;  
    struct Node* next;  
};
```

- data: 4 bytes
- next: 8 bytes (on 64-bit systems)
- **Total per node: 12 bytes + possible padding**

- **Trees and Graphs:** These are hierarchical or network-like structures where data elements (nodes/vertices) are connected via relationships (edges). Each node/vertex stores its data and references to its children or adjacent nodes. Implemented using nodes with multiple pointers (e.g., left and right for binary trees). Stored in non-contiguous dynamic memory.

- Each node holds:
 - **Data**
 - **One or more pointers** (e.g., left, right, neighbours)

- **Stacks and Queues:** These linear data structures can be implemented using arrays or linked lists. Their storage representation depends on the underlying implementation.

- For example, a **stack** can be an array where elements are pushed and popped from one end (top). The order may be **LIFO (Last In First Out)** or **FILO (First In Last Out)**.

- **Push:** Adds a new element on the stack.
- **Pop:** Removes and returns the top element from the stack.
- **Peek:** Returns the i^{th} element from the top of the stack.
- **Change:** Updates the i^{th} element from the top of the stack.

- A **queue** is a linear data structure that follows the **First-In, First-Out (FIFO)** principle. This means that the element inserted first into the queue will be the first one to be removed, similar to a line of people waiting for a service.

- Queue uses two pointers –
- **Front:** The front pointer accesses the data from the front end (helping in enqueueing)
- **Rear:** the rear pointer accesses data from the rear end (helping in dequeuing).



- **Heap memory allocation**
- **Dynamic Allocation:** Non-primitive data structures are usually allocated in heap memory (also called dynamic memory). Heap memory is a large pool of memory that can be dynamically allocated and deallocated at runtime, allowing these structures to expand or shrink as needed.
- **References:** Instead of storing the actual values, non-primitive variables store references (memory addresses) that point to the location where the actual data or object is stored in the heap.



GTU Questions from Unit 1

Sr. No	Questions	CO	Answer	Marks
1	Discuss various types of data structures with example. [NLJIET]	1	Pg 1,2,3,4,5	3
2	Differentiate between data types and data structures. [NLJIET] What is data structure? Differentiate between data types and data structures [NLJIET]	1	Pg 1,6,7	3
3	Explain primitive and Non-primitive data types in detail. [NLJIET]	1	Pg 1,2,3,4,5	3
4	Differentiate Primitive and Non Primitive Data Structures. [NLJIET] Differentiate primitive and non-primitive data structures. [NLJIET]	1	Pg 5,6	3
5	What is DS? Explain data structure and its types. [NLJIET] Define data structure. Enlist and explain types of data structure. [NLJIET] Explain data structure. Enlist and explain the importance of data structure. [NLJIET] Define data structure. What are the applications of data structures? [NLJIET]	1	Pg 1,2,3,4,5	3
6	What is the difference between linear and non-linear data structures? (Dec-2025) [NLJIET] Differentiate between Linear and Non-Linear Data Structures. (Jun-2026) [NLJIET]	1	Pg 6	3
7	Answer the followings: (1) Give examples of Linear and Non-Linear Data Structures. (2) What do you mean by Abstract Data Types? [NLJIET]	1	Pg 2,3,4,5, 7,8	4
8	Compare primitive and non primitive data types. datastructures. [NLJIET] Differentiate between primitive and non-primitive data structures with examples. (Dec-2025) [NLJIET]	1	Pg 5,6	4
9	Explain linear and Non-linear data structure with example. [NLJIET] Explain types of data structures with examples.	1	Pg 1,2,3,4,5	4



New L.J Institute of Engineering and Technology
Semester: III Subject: Data Structures (BE03000081)
Unit 1: Introduction to Data Structures

	[NLJIET] Define static, dynamic, linear and non-linear data structures. [NLJIET]			
10	Differentiate primitive and non-primitive data structures. [NLJIET]	1	Pg 5,6	4
11	Explain storage representation of Primitive and Non Primitive Data Structures . (Jun-2026) [NLJIET]	1	Pg 8,9, 10,11,12	4
12	Explain following: (i) primitive data structures (iii) non-primitive data structures (iv) linear data structures (v) nonlinear data structures [NLJIET]	1	Pg 1,2,3,4,5	7
13	Explain storage representation methods for different data structures and discuss their impact on algorithm implementation and efficiency. (Dec-2025) [NLJIET]	1	Pg 8,9,10, 11,12	7